# Robust Surgical Tool Detection in Laparoscopic Surgery Using YOLOv8 Model


1st Hai-Binh Le
*Foreign Language Specialized High School*
*ULIS, Vietnam National University*
Hanoi, Vietnam

2nd Thai Dinh Kim
*International School*
*Vietnam National University*
Hanoi, Vietnam

3th Manh-Hung Ha
*International School*
*Vietnam National University*
Hanoi, Vietnam

4th Anh Long Quang Tran
*International School*
*Vietnam National University*
Hanoi, Vietnam

5th Duy-Thuc Nguyen
*International School*
*Vietnam National University*
Hanoi, Vietnam

6rd Xuan-Minh Dinh
*Faculty of Electrical Engineering*
*Hanoi University of Industry*
Hanoi, Vietnam



***Abstract*—Surgical tool detection involves identifying the position and type of instruments in an image. This is one of the significant issues in automatic video analysis that can aid in evaluating the surgical skills of doctors or automating the process of controlling the viewing angle of the endoscopic camera. This paper presents a robust method for detecting surgical tools using the YOLOv8 model. We trained four different versions of YOLOv8, evaluated their effectiveness, and compared them with previous models. The experimental results indicate that the YOLOv8 models have an average mAP50 greater than 95.6% across all classes, and are significantly better than some previous research findings.**



***Index Terms*—Surgical tool detection, YOLO, CNN.**


## I. Introduction

Today, laparoscopic surgery is gradually replacing the traditional open surgery method due to its outstanding advantages, including less postoperative pain, faster recovery, more minor scars, a lower risk of infection, and a shorter hospital stay [1]. In laparoscopic surgery, doctors make small incisions in the patient's body to allow surgical instruments and an endoscope to pass through. The surgeon then cuts or burns using hand tools while viewing images on a screen powered by a camera mounted on the endoscope. Because viewing the patient's abdominal cavity directly is impossible, the laparoscopic technique is more complicated than traditional open surgery, especially for less experienced doctors [1]. As a result, the training process for laparoscopic surgeons is often quite lengthy, and skill assessment after training is still done manually through observation and subjective evaluation by an expert.

The field of computer vision has made significant advancements, and as a result, it has become increasingly essential to integrate it into computer-assisted interventions (CAI) for laparoscopic surgery [2]. To achieve this, a key area of interest in computer vision involves detecting surgical instruments in images captured by an endoscopic camera. By doing so, an automatic tool can be developed to assess the effectiveness of laparoscopic surgery through instrument tracking [3]. Furthermore, surgical instrument detection can provide feedback on instrument positioning, which can then be used to adjust the endoscopic camera's positioning automatically [4].

Several studies have been conducted on the problem of detecting surgical instruments, which can be categorized as feature-based and deep learning-based methods. Methods based on feature extraction concentrate on tracking the surgical instrument tip, utilizing features such as color [5], gradient [6], and shape [7]. However, this approach has a low detection accuracy for surgical instruments due to the effects of smoke, blood, luminance, the gloss of tissues, and fat when the camera's light enters the patient's abdomen.

Recently, the approaches based on deep learning that have been strongly developed have been applied in recent studies. This approach mainly applies convolutional neural networks (CNN), which typically have three layers: a convolutional layer, a pooling layer, and a fully connected layer (FC). For example, Sarikaya et al. [8] developed a region proposal network (RPN) and multi-modal two-stream convolutional network that jointly perform surgical instrument detection and localization by combining image and temporal motion cues. In [3], a new dataset called m2cai16-tool locations and approach based on region-based convolutional neural networks was presented for spatial tool detection in real-world laparoscopic surgical videos. This method outperforms previous works in frame-level presence detection, runs in real-time at a frame rate of 5fps, and can extract rich surgical assessment metrics. These studies utilized two-stage CNN models or region proposal-based algorithms, resulting in execution speeds that may be too slow for practical real-time applications.

Therefore, Choi et al. utilized one-stage CNN models, specifically the YOLO model [9], to detect surgical instruments [10]. They further improved their work by combining the YOLO9000 model [11] with Motion Vector Prediction to enhance the accuracy of instrument detection while attaining real-time speed [12]. The YOLO models offer the advantage of fast execution speed, and the updated versions aim to enhance both detection accuracy and speed of execution. This paper explores and employs the latest YOLOv8 model [14]

for detecting surgical tools. We train and test our model using the m2cai16-tool locations dataset [3]. Ultimately, we provide both qualitative and quantitative assessments of the precision parameters and compare them with those of previous models.

The rest of the paper is organized as follows: Section II provides an overview of the architecture of the YOLOv8 model, while Section III presents several results from the surgical tool dataset. Lastly, in Section IV, conclusions are drawn.

## II. Method

### A. YOLO series

YOLO (You Only Look Once) is a popular algorithm for object detection that can accurately detect and pinpoint objects in images and videos in real-time. Over the years, several updates to YOLO have been developed with various enhancements and optimizations, which have improved the speed and accuracy of the model while maintaining its effectiveness.

The first version of YOLO, called YOLOv1 (2015) [9], divided an image into a grid and predicted bounding boxes, class probabilities, and confidence scores for each cell. Though it was fast and accurate, it had localization errors and low recall. YOLOv2 (2016) addressed these issues with anchor boxes and skip connections. It also included batch normalization (BN) and multiscale training, which improved accuracy and speed.

YOLOv3 was a significant upgrade, featuring a feature pyramid network (FPN), better anchor box clustering, and a more powerful classifier architecture. It improved speed and accuracy and became widely used. YOLOv4 (2020) [16]was even more advanced, featuring CSPDarknet53, Cross Stage Partial (CSP), Path Aggregation Network (PAN), and more powerful classifiers. It introduced several optimizations for accuracy and speed, making it among the most advanced algorithms for object detection.

In contrast, YOLOv5 [17] leveraged Spatial Pyramid Pooling - Fast (SPPF) and introduced the Focus module for better model performance. YOLOv6 [18] used EfficientRep as the backbone network and introduced Rep-PAN to the Neck. Its Head was modeled after YOLOX [19] using decoupling operations. Finally, YOLOv7 [20] introduced a re-parameterized convolution module and emphasized model scaling and expansion to optimize parameter utilization.

### B. YOLOv8

Ultralytics, the developer of YOLOv5, released YOLOv8 [100] in January 2023. However, as of writing this, no paper on YOLOv8's architecture is available, making it difficult to compare it to other YOLO versions. However, like most recent YOLO releases, YOLOv8 is anchor-free, which reduces the number of box predictions and speeds up Non-maximum suppression (NMS). Also, during training, YOLOv8 uses mosaic augmentation. However, since it can be detrimental to the training process, the model disables the augmentation for the last ten epochs. YOLOv8 can be run through the command line interface (CLI) or by installing a PIP package. It also has various integrations available for labeling, training, and deployment. YOLOv8 provides five scaled versions ranging from YOLOv8n (nano) to YOLOv8x (extra large).

*1) YOLOv8 Architecture:* YOLOv8 architecture consists of Backbone, Neck, Head, and Loss, as shown in Fig. 1. In YOLOv8's backbone, the use of CSP significantly enhances the learning capability of the CNN by dividing the feature map into two parts. The first part applies convolutional operations, while the second part is concatenated with the output of the convolutional operations from the previous section, reducing the computational effort of the network model. Unlike the C3 module, which comprises three ConvModules and n BottleNecks used in YOLOv5 [16], YOLOv8 employs the C2f module that is composed of two ConvModules and n BottleNecks connected through Split and Concat. The ConvModule, consisting of ConvBN-SiLU, is illustrated in Figure 2. The C2f module is the result of combining the C3 module and ELAN concept in YOLOv7, allowing YOLOv8 to acquire more gradient flow information while keeping the algorithm model lightweight. Moreover, YOLOv8 follows the same SPPF module as YOLOv5, which is a significant upgrade of SPP in enhancing the model's inference speed.

Similar to YOLOv5, YOLOv8's neck follows the FPN and PAN structures and removes the convolution operations in the up-sampling stage. The FPN structure is used to increase the amount of feature information in the lower feature map by upsampling from top to bottom. The PAN structure is used to increase the amount of positional information in the upper feature map by downsampling from bottom to top. These two feature outputs are finally merged to ensure accurate predictions for images of different sizes.

Unlike YOLOv5, YOLOv8 uses a different approach by utilizing the decoupled head method to separate the detection and classification head. As shown in Figure 2, YOLOv8 eliminates the objectness branch and only keeps the regression and classification branches. Additionally, YOLOv8 abandoned the Anchor-Based strategy used before and instead opted for the Anchor-Free technique. This technique enables the object to be located by referencing its center and then predicting the distance from the center to the bounding box.

*2) Loss Function:* When assigning positive and negative samples, the YOLOv8 algorithm utilizes the Task Aligned Assigner from TOOD [11]. This means that positive samples are assigned based on a combination of classification and regression-weighted scores, as shown in the following equation:

$$t = s^{\alpha} * u^{\beta} \tag{1}$$

where s represents the expected score for the given labeled class, u indicates the intersection over union (IoU) of the predicted and actual bounding boxes.

The YOLOv8 algorithm consists of two branches for classification and regression. The classification branch employs BCE Loss, which is presented by the equation below:

$$Loss_n = -w\left[y_n \log x_n + (1 - y_n)\log(1 - x_n)\right] \tag{2}$$

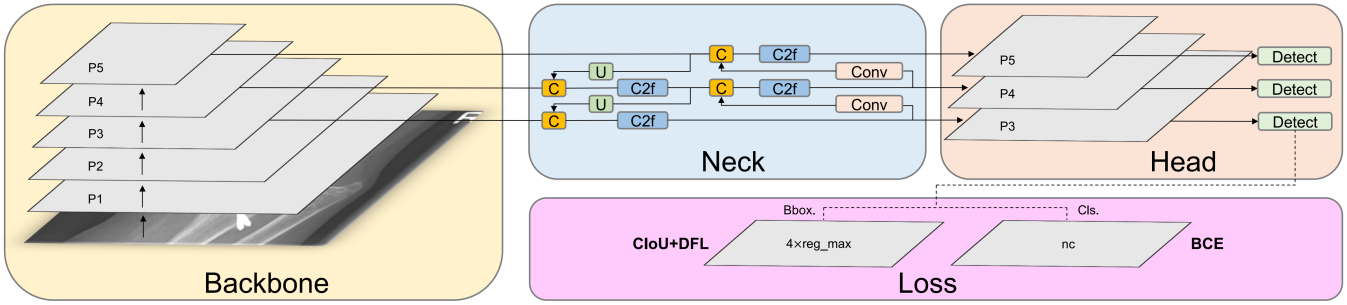



Fig. 1: The overall network model architecture of YOLOv8.

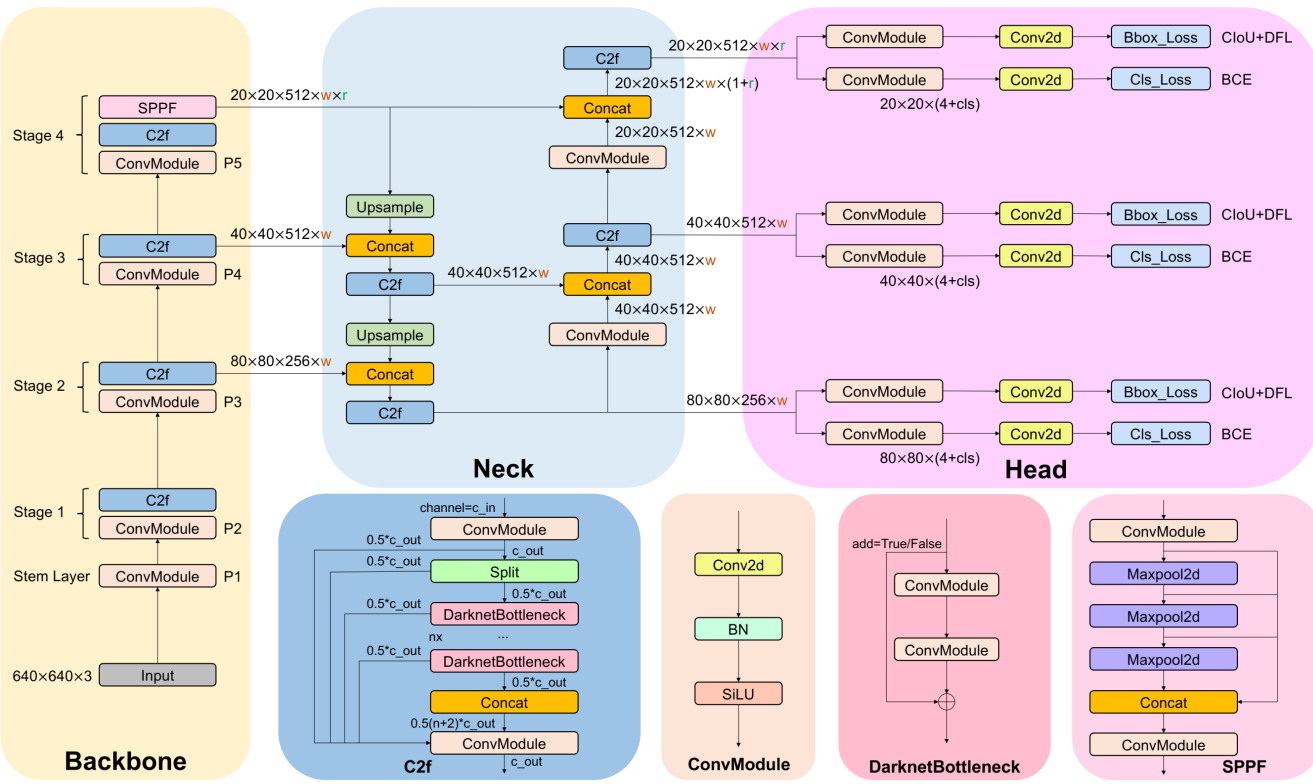



Fig. 2: The detailed network model architecture of YOLOv8.

where $w$ is the weight, $y_n$ is the labeled value, and $x_n$ is the predicted value of the algorithm model.

By incorporating Distribute Focal Loss (DFL) and CIoU Loss, the regression branch prioritizes amplifying the probability distribution around the object y as defined by the DFL equation.

$$DFL(S_n, S_{n+1}) = -\left((y_{n+1} - y)\log S_n + (y - y_n)\log S_{n+1}\right) \tag{3}$$

where

$$S_n = \frac{y_{n+1} - y}{y_{n+1} - y_n}, S_{n+1} = \frac{y_n - y}{y_n - y_{n+1}}$$

The incorporation of the aspect ratio of both prediction and ground truth bounding boxes is illustrated in the CIoU Loss; it enhances the DIoU Loss [52] via an additional influence factor.

$$CIoU\,Loss = 1 - CIoU = 1 - IoU + \frac{d_2^2}{d_C^2} + \frac{v^2}{(1 - IoU) + \nu} \tag{4}$$

where $\nu$ is the parameter to measure the consistency of aspect ratio which is defined as follows:

$$\nu = \frac{4}{\pi^2}(\arctan\frac{w^{gt}}{h^{gt}} - \arctan\frac{w^p}{h^p})^2 \tag{5}$$

## III. Results

### A. Dataset

In order to conduct this study, a sufficiently large dataset was needed which included annotations of names and locations in endoscopic images for instruments. However, at the current stage of research, this dataset is incomplete. As a result, in

this paper, we will be utilizing the m2cai16-tool-locations dataset provided in [3] to train and test the effectiveness of our proposed model.

The m2cai16-tool-locations dataset was developed from the m2cai16-tool dataset [21] specifically for detecting seven instruments used commonly in laparoscopic cholecystectomy, as depicted in Fig. 3. This dataset contains 2811 labeled images with the boxes' names and coordinates surrounding each tool's tip. We use 50%, 30%, and 20% for training, validation, and test splits.

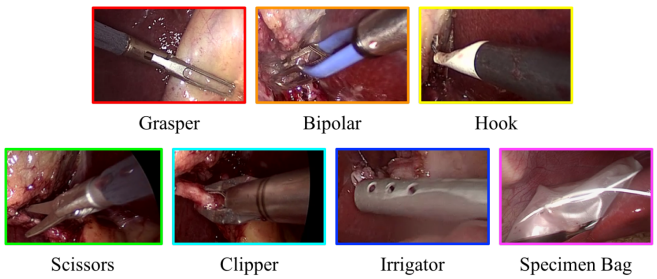



Fig. 3: Seven tools used commonly in laparoscopic cholecystectomy.

Table 1 outlines the number of images, names, and labels for each of the seven surgical tools in the m2cai16-tool-locations dataset. For training, we used recorded data from video-1 through video-7, while recorded data from video-10 was used for validation, and data from video-8 and video-9 were used for testing.

TABLE I: Number of classes and images in datasets

| **Dataset** | ***Training*** | ***Test*** | ***Validation*** |
|---|---|---|---|
| Grasper | 707 | 293 | 422 |
| Hook | 165 | 64 | 79 |
| Clipper | 220 | 64 | 116 |
| Scissors | 197 | 84 | 107 |
| SpecimenBag | 241 | 96 | 139 |
| Bipolar | 215 | 95 | 140 |
| Irrigator | 228 | 84 | 173 |
| Number of images | 1405 | 563 | 843 |

### B. Evaluation Metric

Evaluation metrics used here include: Mean average precision (mAP), average precision (AP), precision (P), and recall (R). The formulas for P and R are as follows:

$$P = \frac{TP}{TP + FP} \tag{6}$$

$$R = \frac{TP}{TP + FN} \tag{7}$$

TP represents the count of accurately predicted bounding boxes, FP denotes the number of incorrectly classified positive samples, and FN indicates the quantity of undetected targets.

The average precision (AP) is the model's average accuracy. The mean average precision (mAP) is the average AP value. K represents the number of categories. The formulas for AP and mAP are as follows:

$$AP = \int_0^1 p(r)dr \tag{8}$$

$$mAP = \frac{1}{K}\sum_{i=1}^{K} AP_i \tag{9}$$

The Mean Average Precision (mAP) is a widely used evaluation metric in object detection tasks that measures the algorithm's performance across all classes.

In this paper, we are evaluating the performance of different algorithms using mAP50 and mAP50-95. The term mAP50 refers to the average precision for all classes at an IoU of 0.5. On the other hand, mAP50-95 refers to the average precision for all classes within a range of IoU from 0.5 to 0.95, with a step size of 0.05. Here, IoU (Intersection over Union) represents the ratio of overlap and union between the predicted bounding box ($B_P$) and ground truth bounding box ($B_G$). It calculates the intersection of these two bounding boxes using the following equation:

$$IoU = \frac{area(B_P \cap B_G)}{area(B_P \cup B_G)} \tag{10}$$

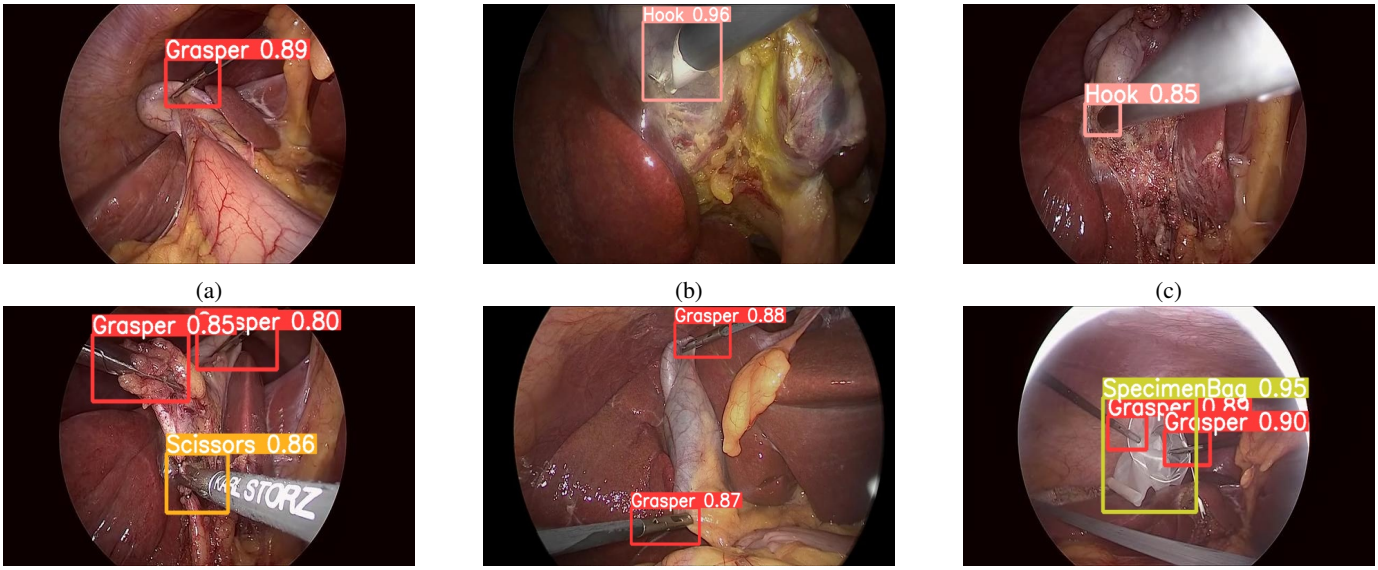
### C. Experimental results

In this study, we trained and tested five YOLOv8 models on the Google Colab platform using pre-trained models that were trained on the COCO val2017 dataset [22]. The parameters for the experimental process in the YOLOv8 models are selected as follows: *epochs=100, patience=50, batch size=16, image size=640, workers=8, optimizer=SGD, lr=0.01, momentum=0.937, weight decay=0.0005.*

We initially set 200 epochs and a patience of 50 using pre-trained models. The training of YOLOv8m stopped at epoch 112, with the best performance observed at epoch 62. To conserve resources, all models were subsequently trained for 100 epochs. Additionally, we use SGD optimizers to train YOLOv8. We set the input image size to 640 and the weight decay of the optimizer is set to 0.0005. The initial learning rate is set to 0.01.

To ensure accurate performance, we first assessed the model qualitatively under various scenarios. These scenarios included single or multiple instruments at varying viewing angles and distances from the camera. Figure 4 depicts some successful detection outcomes using the YOLOv8n model. For instance, in Fig. 4(c), the tip of the hook tool may be partially concealed, yet the model correctly identified the type of tool. However, it is important to note that the YOLOv8n model was unable to detect certain surgical instruments in some instances, as illustrated in Fig. 5.

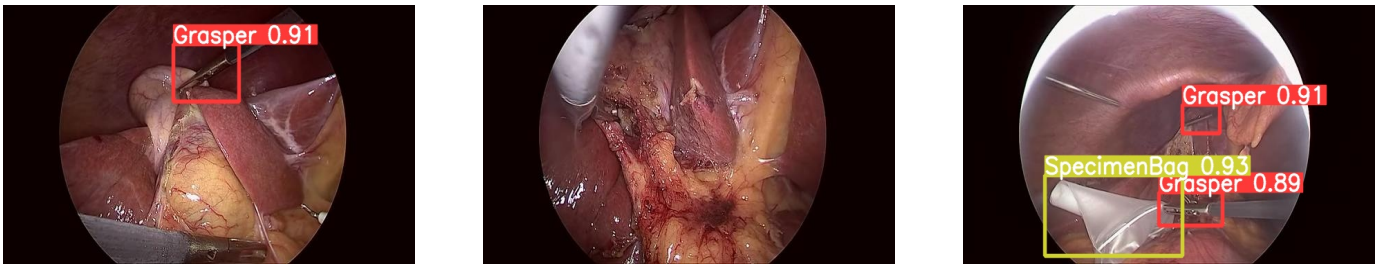
Next, we conducted a quantitative evaluation to determine the effectiveness of the YOLOv8 models on the test dataset. Table II shows that a larger model size has a higher mAP. The YOLOv8l model, comprising 43.3 million parameters, exhibits




(a) (b) (c)

(d) (e) (f)

Fig. 4: Example of surgical tool detection on the test dataset.




(a) (b) (c)

Fig. 5: Example of errors in surgical tool detection on the test dataset.

the highest accuracy, with a mAP50 of 0.965 and a mAP50-95 of 0.636. However, the YOLOv8n model, with only 3.0 million parameters, achieved a mAP50 of 0.956 and an mAP accuracy of 0.607. Therefore, for practical applications, the YOLOv8n model will be deployed on low hardware systems, and YOLOv8l will be recommended for robust hardware systems to achieve the best accuracy.

Table III describes the evaluation parameters for all classes on the test dataset using YOLOv8n. It can be seen that the evaluation indicators for P and R are all greater than 0.9, except for grasper, which has slightly lower values of 0.857 and 0.863, respectively.

TABLE II: Comparison of model performance of different sizes of YOLOv8.

| **Model** | ***params(M)*** | ***P*** | ***R*** | ***mAP50*** | ***mAP50-95*** |
|---|---|---|---|---|---|
| YOLOv8n | 3.0 | 0.93 | 0.929 | 0.956 | 0.607 |
| YOLOv8s | 11.1 | 0.936 | 0.937 | 0.958 | 0.629 |
| YOLOv8m | 25.8 | 0.961 | 0.917 | 0.958 | 0.63 |
| YOLOv8l | 43.6 | 0.969 | 0.91 | 0.965 | 0.636 |

TABLE III: Detailed evaluation results for each class using YOLOv8n model with the input image size of 640

| **Class** | **Instances** | ***P*** | ***R*** | ***mAP50*** | ***mAP50-95*** |
|---|---|---|---|---|---|
| All tools | 780 | 0.93 | 0.929 | 0.956 | 0.607 |
| Grasper | 293 | 0.857 | 0.863 | 0.911 | 0.548 |
| Hook | 64 | 0.974 | 0.969 | 0.982 | 0.736 |
| Clipper | 64 | 0.983 | 0.905 | 0.96 | 0.689 |
| Scissors | 84 | 0.903 | 0.952 | 0.964 | 0.576 |
| SpecimenBag | 96 | 0.914 | 0.917 | 0.947 | 0.63 |
| Bipolar | 95 | 0.953 | 0.958 | 0.976 | 0.582 |
| Irrigator | 84 | 0.929 | 0.94 | 0.953 | 0.491 |

TABLE IV: Comparison model performance using mAP50

| **Class** | **YOLOv8n** | ***YOLOv8l*** | ***K. Jo [12]*** | ***A. Jin [3]*** |
|---|---|---|---|---|
| All tools | 0.956 | 0.965 | 0.847 | 0.818 |
| Grasper | 0.911 | 0.90 | 0.921 | 0.872 |
| Hook | 0.982 | 0.995 | 0.859 | 0.953 |
| Clipper | 0.96 | 0.974 | 0.853 | 0.884 |
| Scissors | 0.964 | 0.975 | 0.821 | 0.708 |
| SpecimenBag | 0.947 | 0.96 | 0.832 | 0.821 |
| Bipolar | 0.976 | 0.97 | 0.823 | 0.751 |
| Irrigator | 0.953 | 0.979 | 0.829 | 0.735 |

Lastly, we compared the efficacy of two models, YOLOv8n and YOLOv8l (a low version and a high version, respectively), with specific models described in a previous paper to evaluate the effectiveness of our proposed solution. As shown in Table IV, the mAP50 values for the YOLOv8n and YOLOv8l models are superior to those proposed in [3], [12].

## IV. Conclusions

In this paper, we have introduced a powerful method for detecting surgical instruments in laparoscopic surgery. The model we used is YOLOv8. The results show that the evaluation parameters, including P, R, and mAP, have high values and are significantly better than some previous models. With a dataset of only 2811 labeled images, the mAP50 for all instruments exceeds 95.6% for the YOLOv8 models. The next research direction is to enhance the YOLOv8 model by adding an attention component and implementing practical applications, such as the evaluation of automatic surgeon skills.

## Acknowledgment

This research is funded by International School, Vietnam National University, Hanoi (VNU-IS).

## References

[1] Kim, D.T., Cheng, C., Liu, D., Liu, K., and Huang, W. (2019). Designing a New Endoscope for Panoramic-View with Focus-Area 3D-Vision in Minimally Invasive Surgery. Journal of Medical and Biological Engineering, 40, 204-219.

[2] Münzer, B., Schöffmann, K., and Böszörményi, L. (2016). Content-based processing and analysis of endoscopic images and videos: A survey. Multimedia Tools and Applications, 77, 1323-1362.

[3] Jin, A., Yeung, S., Jopling, J.K., Krause, J., Azagury, D., Milstein, A., and Fei-Fei, L. (2018). Tool Detection and Operative Skill Assessment in Surgical Videos Using Region-Based Convolutional Neural Networks. 2018 IEEE Winter Conference on Applications of Computer Vision (WACV), 691-699.

[4] Cheng, T., Li, W., Ng, W.Y., Huang, Y., Li, J., Ng, C.S., Chiu, P.W., and Li, Z. (2021). Deep Learning Assisted Robotic Magnetic Anchored and Guided Endoscope for Real-Time Instrument Tracking. IEEE Robotics and Automation Letters, 6, 3979-3986.

[5] Allan, M., Ourselin, S., Thompson, S.A., Hawkes, D.J., Kelly, J.D., and Stoyanov, D. (2013). Toward Detection and Localization of Instruments in Minimally Invasive Surgery. IEEE Transactions on Biomedical Engineering, 60, 1050-1058.

[6] Rieke, N., Tan, D.J., Alsheakhali, M., Tombari, F., Filippo, C.A., Belagiannis, V., Eslami, A., and Navab, N. (2015). Surgical Tool Tracking and Pose Estimation in Retinal Microsurgery. International Conference on Medical Image Computing and Computer-Assisted Intervention.

[7] Sznitman, R., Richa, R., Taylor, R.H., Jedynak, B.M., and Hager, G. (2013). Unified Detection and Tracking of Instruments during Retinal Microsurgery. IEEE Transactions on Pattern Analysis and Machine Intelligence, 35, 1263-1273.

[8] Sarikaya, D., Corso, J.J., and Guru, K.A. (2017). Detection and Localization of Robotic Tools in Robot-Assisted Surgery Videos Using Deep Neural Networks for Region Proposal and Detection. IEEE Transactions on Medical Imaging, 36, 1542-1549.

[9] Redmon, J., Divvala, S.K., Girshick, R.B., and Farhadi, A. (2015). You Only Look Once: Unified, Real-Time Object Detection. 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 779-788.

[10] Choi, B., Jo, K., Choi, S., and Choi, J. (2017). Surgical-tools detection based on Convolutional Neural Network in laparoscopic robot-assisted surgery. 2017 39th Annual International Conference of the IEEE Engineering in Medicine and Biology Society (EMBC), 1756-1759.

[11] Redmon, J., and Farhadi, A. (2016). YOLO9000: Better, Faster, Stronger. 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 6517-6525.

[12] Jo, K., Choi, Y., Choi, J., and Chung, J.W. (2019). Robust Real-Time Detection of Laparoscopic Instruments in Robot Surgery Using Convolutional Neural Networks with Motion Vector Prediction. Applied Sciences.

[13] Boonkong, A., Hormdee, D., Sonsilphong, S., and Khampitak, K. (2022). Surgical Instrument Detection for Laparoscopic Surgery using Deep Learning. 2022 19th International Conference on Electrical Engineering/Electronics, Computer, Telecommunications and Information Technology (ECTI-CON), 1-4.

[14] Ultralytics yolov8. https://github.com/ultralytics/ultralytics (2023)

[15] Redmon, J., and Farhadi, A. (2018). YOLOv3: An Incremental Improvement. ArXiv, abs/1804.02767.

[16] Bochkovskiy, A., Wang, C., and Liao, H.M. (2020). YOLOv4: Optimal Speed and Accuracy of Object Detection. ArXiv, abs/2004.10934.

[17] Ultralytics yolov5. https://github.com/ultralytics/yolov5 (2022).

[18] Li, C., Li, L., Jiang, H., Weng, K., Geng, Y., Li, L., Ke, Z., Li, Q., Cheng, M., Nie, W., Li, Y., Zhang, B., Liang, Y., Zhou, L., Xu, X., Chu, X., Wei, X., and Wei, X. (2022). YOLOv6: A Single-Stage Object Detection Framework for Industrial Applications. ArXiv, abs/2209.02976.

[19] Ge, Z., Liu, S., Wang, F., Li, Z., and Sun, J. (2021). YOLOX: Exceeding YOLO Series in 2021. ArXiv, abs/2107.08430.

[20] Wang, C., Bochkovskiy, A., and Liao, H.M. (2022). YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors. ArXiv, abs/2207.02696.

[21] Twinanda, A.P., Mutter, D., Marescaux, J., Mathelin, M.D., and Padoy, N. (2016). Single- and Multi-Task Architectures for Tool Presence Detection Challenge at M2CAI 2016. ArXiv, abs/1610.08851.

[22] Lin, Tsung-Yi, et al. "Microsoft coco: Common objects in context." Computer Vision–ECCV 2014: 13th European Conference, Zurich, Switzerland, September 6-12, 2014, Proceedings, Part V 13. Springer International Publishing, 2014.